

trading-portal

Project documentation — generated from README.md

Source last updated: 2026-09-29 17:10 · PDF generated: 2026-09-29 17:10 · release: v1.1.0

trading-portal

BalRam's live ETF portal (QQQ, SPY, GLD) — served by GitHub Pages.

Created 2026-09-28 · Last updated 2026-09-29

Files

- `etf-live.html` — the portal page. On every load/refresh it fetches fresh quotes from the portal worker's `/api/quotes` endpoint (server-side Yahoo fetch, 60-second edge cache); the baked bundle below remains as the instant first-paint fallback if the API is unreachable.

- `etf-live.css` — its stylesheet
- `etf-live-bundle.json` — market-data snapshot baked server-side into the page and the sibling JSON. The 5-minute rebake cron was retired 2026-09-29 when the page moved to the live `/api/quotes` feed; the bundle is now a first-paint fallback only.
- `nyse-holidays.json` — the NYSE full-day closure list the page uses for session labels; assembled at deploy time from the cross-thread repo (see below), never fetched by visitors from GitHub

The page fetches live quotes from the portal worker's `/api/quotes` endpoint on every load/refresh and falls back to the baked bundle for instant first paint. It loads the holiday file from the same deployed origin before first paint.

Branches

- `main` — the source of truth: page source, styles, tests, deploy script.
- `gh-pages` — the generated, deployed branch: page source from `main` plus the assembled `etf-live-bundle.json` and `nyse-holidays.json`. Regenerated on every deploy; never edited by hand.

Holiday data flow

The page's session labels depend on the NYSE holiday list. The canonical list is maintained quarterly by the `holiday-list-update` job and mirrored to `holidays/nyse_holidays.json` in `technology-consults/cross-thread-task-status-tracker`. Deployments assemble the portal's `nyse-holidays.json` from that git copy — never from a web search, and never fetched live by the page from GitHub.

Deploying

`scripts/deploy_portal.py` builds the deployable and pushes the `gh-pages` branch. Deploy policy: the portal's own files are hard requirements (any read/write failure aborts the deploy); the holiday JSON is best-effort — if it cannot be fetched or fails validation (structure + freshness), the previously deployed copy

is carried forward untouched and the deploy continues with a warning, so a dead holiday pipeline never blocks shipping a page fix. `docs/` is excluded from the served branch. `tests/` holds the portal regression suite, which must pass before any deploy (holiday-data checks are advisory warnings; only portal-file checks can fail the gate).

Releases are tagged on `main` with semver (e.g. `v1.0.0`) before deploying, so the tag always identifies the exact deployed source commit. Every deploy prepares a backout package; the policy is fix-forward, rollback only as a last resort.